

CL-AWS2 — AWS avancés + serverless

Jour 2 — Données et découplage : DynamoDB, événements S3, SQS, SNS

Hier l'API répondait des données en dur. Aujourd'hui, une vraie base — sans serveur non plus.

Objectifs de la journée

À la fin de cette journée, vous serez capable de :

- **Expliquer** le modèle clé-valeur de DynamoDB : table, clé de partition, clé de tri, capacité à la demande.
- **Utiliser** les opérations `put_item`, `get_item`, `query` avec boto3 — et dire pourquoi on évite `scan`.
- **Concevoir** la table DynamoDB de StockLine (produits + mouvements) à partir des schémas d'accès.
- **Déclencher** une Lambda sur un dépôt de fichier S3 et lire l'événement reçu.
- **Choisir** entre SQS (file) et SNS (pub/sub) et expliquer visibilité, DLQ et fan-out.
- **Assembler** le pipeline complet : CSV → S3 → Lambda → DynamoDB → SNS.

Fin de journée : mini-TP (non noté) — vous construisez ce pipeline de bout en bout.

Plan de la journée

1. **DynamoDB** — le modèle clé-valeur, les clés, la capacité, les opérations, boto3.
2. **StockLine sur DynamoDB** — concevoir la table à partir des accès réels.
3. **Événements S3** — du fichier déposé au code exécuté.
4. **SQS** — files, visibilité, DLQ : pourquoi découpler.
5. **SNS** — pub/sub et fan-out.

Démo : le pipeline d'ingestion complet (🖥️ 2-1). Exercices : concevoir des clés DynamoDB (✏️ 2-1), choisir SQS ou SNS (✏️ 2-2). Mini-TP de fin de journée (🧑‍🎓, non noté) : le pipeline, par vous, avec teardown.

Où en est StockLine ?

Hier soir :

- `GET /produits` répond derrière API Gateway... avec **3 produits en dur** dans le code.
- `POST /produits` renvoie **501** : « une fonction est éphémère, rien ne survit ».

Le problème est exactement celui du TP1 → TP2 : il faut un **stockage durable, partagé entre toutes les exécutions**. Au TP2, c'était RDS PostgreSQL. Mais :

- RDS = une instance **facturée à l'heure**, même la nuit — on retomberait dans le problème d'hier matin.
- RDS vit dans un VPC : chaque Lambda devrait s'y attacher (complexité réseau, cold starts plus longs).

Il nous faut une base **au même modèle économique que Lambda** : payée à la requête, scalée par AWS, sans instance. C'est **DynamoDB**.

1. DynamoDB

La base de données du serverless

Clé-valeur vs relationnel : deux philosophies

	PostgreSQL (TP2)	DynamoDB
Modèle	tables, colonnes fixes, jointures	items à schéma flexible, pas de jointure
Requêtes	SQL — n'importe quelle question	par clé — les questions prévues à l'avance
Schéma	défini avant les données	seule la clé est imposée
Scaling	instance plus grosse (verticale)	horizontal, automatique, illimité
Facturation	à l'heure (instance)	à la requête + stockage
Latence	variable avec la charge	quelques ms, stable

Le contrat est inversé : SQL vous laisse poser toute question mais vous gérez la machine ; DynamoDB gère tout **si** vous connaissez vos questions d'avance. D'où la règle du jour : **on conçoit la table à partir des requêtes, pas des données.**

Table, items, attributs

- Une **table** DynamoDB = un ensemble d'**items** (≈ lignes).
- Un **item** = un ensemble d'**attributs** (≈ colonnes), **jusqu'à 400 Ko**.
- Seuls les attributs de **clé** sont obligatoires et typés à la création. Tout le reste est libre : deux items de la même table peuvent avoir des attributs différents.

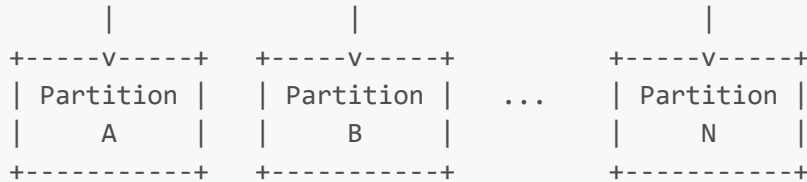
```
{ "pk": "PRODUIT", "sk": "SSD-500", "nom": "Disque SSD 500 Go",
  "prix_unitaire": 59.90, "seuil_alerte": 5 }

{ "pk": "MVT#SSD-500", "sk": "2026-07-02T09:15:00+00:00#a1b2c3d4",
  "type": "entree", "quantite": 42 }
```

Pas de **CREATE TABLE** avec 15 colonnes : on déclare **pk** et **sk**, et c'est tout. La souplesse est réelle — la discipline devient **votre** responsabilité (validation dans le code, comme hier dans **lambda_ingestion**).

La clé de partition : où vit l'item

```
put_item(pk="PRODUIT", ...)      la valeur de pk est hachée
    |                           pour choisir la partition
    v
+-----+ hash(pk) -----+
```



- La **clé de partition** (*partition key*, HASH) détermine **sur quelle machine** l'item est rangé.
- Accès par clé exacte = direct, quelques millisecondes, **quelle que soit la taille de la table**.
- Corollaire : une bonne clé de partition **répartit** les accès. Si tout le trafic tape la même valeur → *hot partition*, throttling.

La clé de tri : l'ordre dans la partition

Une table peut avoir une clé **composite** : partition (**pk**) + **tri** (**sk**, RANGE).

- Tous les items d'une même **pk** sont stockés **triés par sk**.
- On peut alors demander : « tous les items de cette partition », « ceux dont **sk** commence par... », « ceux entre deux valeurs » — c'est l'opération **Query**.

```
pk = "MVT#SSD-500"           (la partition du produit)
sk = "2026-07-01T08:00:00+00:00#3f2a"  type=entree  quantite=42
sk = "2026-07-01T14:30:00+00:00#9c1e"  type=sortie  quantite=5
sk = "2026-07-02T09:15:00+00:00#a1b2"  type=sortie  quantite=3
                                ^ triés chronologiquement, gratuitement
```

Un horodatage ISO 8601 en clé de tri = historique trié sans **ORDER BY**. C'est LE pattern à retenir pour des mouvements, des logs, des mesures.

Capacité : à la demande (notre choix) ou provisionnée

DynamoDB mesure le travail en unités : **RCU** (lecture ≤ 4 Ko) et **WCU** (écriture ≤ 1 Ko).

Mode	Principe	Quand
À la demande (PAY_PER_REQUEST)	payé à la requête, scaling instantané	trafic inconnu ou irrégulier — notre cas
Provisionné	on réserve N RCU/WCU par seconde	trafic stable et connu, moins cher à volume constant

Coûts à la demande (eu-west-3, ordre de grandeur) : ~0,25 € le **million** de lectures, ~1,25 € le million d'écritures, stockage ~0,25 €/Go/mois.

💰 Free-tier permanent : **25 Go de stockage** gratuits, et 25 RCU + 25 WCU en mode provisionné. Nos 3 jours + le TP3 coûteront **quelques centimes** au pire.

Les opérations : put_item et get_item

Écriture et lecture **par clé complète** — les deux opérations reines :

```
import boto3
from decimal import Decimal

table = boto3.resource("dynamodb").Table("abc-stockline")

# Écrire (crée OU remplace l'item entier ayant cette clé)
table.put_item(Item={
    "pk": "PRODUIT",
    "sk": "SSD-500",
    "nom": "Disque SSD 500 Go",
    "prix_unitaire": Decimal("59.90"),  # jamais de float !
    "seuil_alerte": 5,
})

# Lire UN item par sa clé complète (pk + sk)
resultat = table.get_item(Key={"pk": "PRODUIT", "sk": "SSD-500"})
item = resultat.get("Item")  # None si absent -> .get, pas ["Item"]
```

Deux pièges Python : DynamoDB **refuse les float** (→ `Decimal(str(x))`) et renvoie les nombres en `Decimal` (→ encodeur JSON personnalisé, vu dans `lambda_produits.py`).

Query : interroger UNE partition

```
from boto3.dynamodb.conditions import Key

# Tous les produits (la partition "PRODUIT")
resultat = table.query(
    KeyConditionExpression=Key("pk").eq("PRODUIT")
)

# Les mouvements du SSD-500 depuis le 1er juillet
resultat = table.query(
    KeyConditionExpression=Key("pk").eq("MVT#SSD-500")
    & Key("sk").gte("2026-07-01"),
)
for mvt in resultat["Items"]:
    print(mvt["type"], mvt["quantite"])
```

- **Query** exige l'**égalité sur pk** ; sur **sk** : `eq`, `begins_with`, `between`, `gt`, `lt`...
- Coût proportionnel **aux données lues**, pas à la taille de la table.
- Résultats paginés (1 Mo max par page) : boucler sur `LastEvaluatedKey` — le code complet est dans `lambda_produits.py` (`_ddb_get_stock`).

Scan : pourquoi il faut s'en méfier

`scan` lit **toute la table**, partition par partition, puis filtre éventuellement :

```
resultat = table.scan() # lit TOUT. Toujours. Même avec FilterExpression.
```

- Le `FilterExpression` s'applique **après** la lecture : vous payez la lecture de 10 Go pour récupérer 3 items.
- Sur la table de démo (10 items), `scan` répond en 20 ms — « ça marche ». Sur la table de production (10 M d'items), il coûte cher, sature la capacité et peut prendre des minutes. **Le piège est qu'il marche en démo.**

Règles :

1. `scan` interdit dans un chemin d'API (requête utilisateur).
2. Toléré pour un besoin **administratif ponctuel** (audit, export) — de préférence la nuit.
3. Si vous avez « besoin » d'un scan récurrent, c'est que la **conception de la clé** est à revoir (ou qu'il faut un index secondaire — notion survolée slide suivante).

boto3 : client vs resource, et les index en une slide

Deux interfaces `boto3` pour DynamoDB :

- `boto3.client("dynamodb")` : bas niveau, types explicites (`{"S": "SSD-500"}`) — ce que manipule la CLI.
- `boto3.resource("dynamodb").Table(...)` : haut niveau, types Python natifs — **notre choix** dans tout le code de la semaine.

Index secondaires (survol — hors périmètre TP3) :

- Un **GSI** (*Global Secondary Index*) = une copie de la table réorganisée selon **une autre clé**, maintenue par DynamoDB, interrogeable par `Query`.
- C'est la réponse propre à « je voudrais chercher par un autre attribut sans scanner ».
- À retenir pour la certification : GSI = autre clé de partition ; LSI = même partition, autre tri.

2. StockLine sur DynamoDB

Concevoir la table à partir des questions, pas des données

Étape 1 : lister les schémas d'accès

En relationnel (TP2), on modélisait les **entités** : table `produits`, table `mouvements`, clé étrangère. En DynamoDB, on part des **questions que l'API pose** :

#	Question (endpoint)	Fréquence
A1	lister tous les produits — <code>GET /produits</code>	élevée
A2	lire un produit par référence — <code>GET /produits/{ref}</code>	élevée
A3	créer un produit / un mouvement — <code>POST ...</code>	moyenne
A4	tous les mouvements d'un produit, triés par date	moyenne
A5	stock actuel d'un produit — <code>GET /stocks/{ref}</code>	élevée

Chaque question doit trouver sa réponse par **get_item** ou **query** — jamais scan. C'est la contrainte qui va dessiner nos clés.

Étape 2 : la conception retenue

Une seule table `$PREFIX-stockline`, clé composite `pk + sk` :

Item	pk	sk	Attributs
Fiche produit	<code>PRODUIT</code>	<code><reference></code>	nom, prix_unitaire, seuil_alerte
Mouvement	<code>MVT#<reference></code>	<code><horodatage ISO>#<uuid></code>	type, quantite, commentaire

Vérification contre les schémas d'accès :

- **A1** lister les produits → `query(pk = "PRODUIT")` ✓
- **A2** un produit → `get_item(pk="PRODUIT", sk=ref)` ✓
- **A4** mouvements triés → `query(pk = "MVT#<ref>")` — triés par `sk` = par date ✓
- **A5** stock → même query + somme des quantités signées ✓

Le `#uuid` dans la `sk` du mouvement : deux mouvements à la même seconde ne s'écrasent pas (rappel : `put_item` remplace l'item de même clé).

Étape 3 : assumer les limites de cette conception

Une conception honnête se juge aussi par ce qu'elle **ne** fait pas bien :

- Tous les produits partagent la partition `PRODUIT` → acceptable pour un **catalogue** (quelques milliers de références, lecture-majoritaire), pas pour des millions d'items à fort débit d'écriture (*hot partition*).
- « Chercher un produit par nom » → pas de clé pour ça : il faudrait un **GSI**. Au TP3, ce besoin n'existe pas — on ne le paie donc pas.
- Le stock est **recalculé** à chaque `GET /stocks` (query + somme). Alternative : maintenir un compteur sur la fiche produit (`update_item` atomique) — plus rapide en lecture, plus délicat en écriture. Les deux se défendent ; nous gardons le calcul, fidèle au StockLine d'origine.

Moralité : en DynamoDB on **choisit ses compromis à l'avance**. C'est exactement l'exercice 2-1.

Le code d'hier bascule sur DynamoDB

Hier, `GET /produits` en mode mémoire ; aujourd'hui, la **même fonction déployée** passe en mode DynamoDB — uniquement parce que `TABLE_NAME` apparaît dans l'environnement :

```
TABLE_NAME = os.environ.get("TABLE_NAME", "")
_table = boto3.resource("dynamodb").Table(TABLE_NAME) if TABLE_NAME else None
# ...
def _ddb_lister_produits():
    resultat = _table.query(KeyConditionExpression=Key("pk").eq("PRODUIT"))
    return reponse(200, [ ... for item in resultat["Items"]])
```

```
./deploy-api.sh --avec-dynamodb      # met TABLE_NAME + insère 3 produits
curl $URL/produits                  # mêmes données... depuis la table
curl -X POST $URL/produits -H 'content-type: application/json' \
  -d '{"reference":"ECRAN-27","nom":"Écran 27\\\\" , "prix_unitaire":249.0}'
curl $URL/produits/ECRAN-27         # 200 -> le POST 501 d'hier est guéri
```

Sans oublier le rôle : la politique gagne `dynamodb:GetItem/PutItem/Query` **sur cette table uniquement**.



Exercice 2-1 — Concevoir des clés DynamoDB

`exercices/08-cl-aws2/exercice-2-1-conception-cles.md` — 45 min

Trois cas à modéliser (clé de partition, clé de tri, justification par les schémas d'accès) :

1. **Journaux d'audit** d'une flotte de serveurs — « les événements du serveur X sur les dernières 24 h ».
2. **Capteurs de température** de 3 entrepôts — « les mesures du capteur Y ce mois-ci » et « toutes les mesures de l'entrepôt Z aujourd'hui ».
3. **Sessions utilisateur** d'un site — « la session T » et purge automatique après 30 min (indice : cherchez « TTL DynamoDB »).

Pour chaque cas, vous devez aussi désigner **la requête impossible** sans index secondaire — savoir dire non fait partie de la conception.

3. Événements S3

Le fichier déposé qui déclenche du code

S3 → Lambda : le déclencheur asynchrone type

S3 peut émettre une **notification d'événement** à chaque opération sur les objets : création (`s3:ObjectCreated:*`), suppression, restauration... Cible : Lambda, SQS ou SNS.

```
aws s3 cp inventaire.csv s3://bucket/entrants/
    |
    v
    filtre : prefix="entrants/" suffix=".csv"
[ bucket S3 ] --- événement ---> [ Lambda ingestion ] (asynchrone)
    |
    2 retries automatiques en cas d'échec
```

Cas d'usage classiques d'admin cloud :

- **Ingestion** : CSV/JSON déposé → chargé en base (notre pipeline du jour).
- **Miniatures** : image uploadée → redimensionnée dans un autre préfixe.
- **Analyse** : log ou export déposé → parsé, indexé, alerté.

Au TP2, ce besoin aurait été un **cron qui scrute un répertoire**. Ici : zéro polling, zéro machine, latence de quelques secondes.

L'événement S3 reçu par la Lambda

```
{
  "Records": [
    {
      "eventSource": "aws:s3",
      "eventName": "ObjectCreated:Put",
      "s3": {
        "bucket": { "name": "abc-stockline-ingestion-123456789012" },
        "object": { "key": "entrants/inventaire+juillet.csv", "size": 512 }
      }
    }
  ]
}
```

```
for record in event["Records"]:
    bucket = record["s3"]["bucket"]["name"]
    cle = unquote_plus(record["s3"]["object"]["key"]) # décoder l'URL !
    objet = s3.get_object(Bucket=bucket, Key=cle)
```

Trois points de vigilance : **Records** est une **liste** (boucler) ; la clé est **encodée URL** (+, %C3%A9... → **unquote_plus**) ; l'événement ne contient **pas** le fichier — juste de quoi aller le chercher (**s3:GetObject** dans le rôle !).

Deux pièges S3 → Lambda à connaître avant la démo

1. La boucle infinie (classique et coûteuse) Une Lambda déclenchée par le bucket **qui écrit dans le même bucket** se re-déclenche elle-même... à l'infini. Parade : des **préfixes disjoints** — déclencheur filtré sur

`entrants/`, écritures dans `archives/` — ou deux buckets. C'est pour ça que notre pipeline filtre `prefix=entrants/`, `suffix=.csv`.

2. L'ordre permission → notification S3 vérifie, **au moment où vous enregistrez la configuration de notification**, qu'il a le droit d'invoquer la fonction. Il faut donc : `aws lambda add-permission --principal s3.amazonaws.com ... avant put-bucket-notification-configuration` — sinon : `Unable to validate the following destination configurations`.

Et un rappel réseau : le bucket doit être **dans la même région** que la fonction pour les notifications directes (eu-west-3 partout, comme toujours).

4. SQS

Découpler : la file d'attente entre producteur et consommateur

Pourquoi découpler ?

Imaginez le magasin qui dépose **200 CSV d'inventaire d'un coup** à la fermeture :

- **Couplage direct** (appel synchrone) : si le consommateur est lent ou en panne, le producteur échoue aussi. Le pic de charge frappe tout le monde en même temps.
- **Découplage par file** : le producteur dépose ses messages et repart ; le consommateur traite **à son rythme** ; une panne du consommateur ne perd rien — les messages attendent.

```
producteur --> [ file SQS ] --> consommateur
(rapide)      absorbe le      (à son rythme)
              pic, garde
              les messages
```

C'est le même réflexe que l'ASG du TP2 (absorber la charge), mais **sans machine** : SQS est un service managé, facturé au million de requêtes (💰 free-tier permanent : 1 M/mois).

Visibility timeout : le cœur du fonctionnement



```

(traitement fini)      |
      |                | le message REDEVIENT
      |                | visible -> retraité
message parti

```

Un message n'est **pas supprimé à la lecture** : le consommateur doit le supprimer explicitement après traitement. D'où la règle : **SQS garantit « au moins une fois »** — votre traitement doit tolérer les doublons (idempotence).

DLQ : la file des messages maudits

Un message dont le traitement échoue **revient** dans la file... et échoue encore. Sans garde-fou, il tourne pour toujours et bloque l'analyse.

La **DLQ** (*Dead-Letter Queue*) est une seconde file où SQS déplace tout message reçu plus de **maxReceiveCount** fois (ex : 3) sans avoir été supprimé :

```

[ file principale ] --- 3 échecs ---> [ DLQ ]
                                   |
                               alarme CloudWatch + analyse humaine

```

Réflexes d'admin :

- **Toujours** une DLQ sur une file de production + **alarme** sur sa profondeur (**ApproximateNumberOfMessagesVisible** > 0).
- La DLQ n'est pas une poubelle : c'est une **salle d'examen** — on y lit le message fautif, on corrige, on rejoue.
- Lambda asynchrone (S3, SNS...) peut aussi avoir une *destination on-failure* — même idée.

SQS + Lambda : le consommateur sans serveur

Lambda peut consommer une file via un **event source mapping** : le service Lambda interroge la file pour vous et invoque la fonction avec des **lots** :

```

def handler(event, context):
    for record in event["Records"]:          # jusqu'à 10 messages par lot
        message = json.loads(record["body"])
        traiter(message)

```

```

aws lambda create-event-source-mapping \
  --function-name $PREFIX-traitements \
  --event-source-arn arn:aws:sqs:eu-west-3:ACCOUNT_ID:$PREFIX-mouvements \
  --batch-size 10 --region eu-west-3

```

- Si le handler lève une exception, **tout le lot** redevient visible (d'où, encore, l'idempotence). Réglage fin possible par `ReportBatchItemFailures`.
- Le rôle de la fonction doit avoir `sqs:ReceiveMessage`, `sqs:DeleteMessage`, `sqs:GetQueueAttributes`.

5. SNS

Pub/sub : un événement, plusieurs destinataires

SNS : topics et abonnements

SNS (*Simple Notification Service*) inverse la logique de SQS : un message publié sur un **topic** est **poussé immédiatement à tous les abonnés**.

```
publish --> [ topic ]
               +--> abonné email      (l'équipe magasin)
               +--> abonné Lambda     (traitement)
               +--> abonné SQS        (tampon pour un autre service)
               +--> abonné HTTPS      (webhook)
```

- Le publieur ne connaît **pas** les abonnés : on ajoute un destinataire sans toucher au code qui publie.
- Pas de stockage : un message publié sans abonné est **perdu** (contrairement à SQS).
- Abonnement email : à **confirmer** par clic dans le mail — l'oubli n°1 en TP.

💰 Free-tier permanent : 1 M de publications/mois ; 1 000 emails/mois.

SQS vs SNS : le tableau à connaître

	SQS (file)	SNS (pub/sub)
Modèle	<i>pull</i> — le consommateur vient chercher	<i>push</i> — le topic pousse
Destinataires d'un message	un seul consommateur le traite	tous les abonnés le reçoivent
Stockage	oui — jusqu'à 14 jours	non — perdu si aucun abonné
Sert à	absorber la charge, lisser, garantir	diffuser une information
Question à se poser	« qui fait le travail ? »	« qui doit être au courant ? »

Et le meilleur des deux mondes : le **fan-out**.

```
événement -> [SNS]
               +--> [ SQS équipe factu ] --> Lambda facturation
               +--> [ SQS équipe stock ] --> Lambda réappro
               +--> email superviseur
```

Chaque équipe a **sa** file (son rythme, sa DLQ) ; l'événement n'est publié qu'une fois.

Exercice 2-2 — SQS ou SNS ?

[exercices/08-cl-aws2/exercice-2-2-sqs-ou-sns.md](#) — 30 min

Six scénarios d'entreprise, et pour chacun : **SQS, SNS, ou les deux (fan-out)** — avec justification en une phrase, et les réglages qui comptent (DLQ ? visibilité ? confirmation d'abonnement ?).

Extraits :

- « Chaque commande doit être traitée exactement par un des 4 workers, même en cas de pic. »
- « Quand un produit passe sous son seuil d'alerte, prévenir l'appli mobile, l'équipe achats **et** archiver l'événement. »
- « Les rapports PDF prennent 8 minutes à générer ; les demandes arrivent par rafales. »

Un des scénarios est un piège : il ne relève **ni** de SQS **ni** de SNS. Trouvez-le — la réponse arrive demain matin (indice : J3).



Démo 2-1

Le pipeline complet : CSV → S3 → Lambda → DynamoDB → SNS

[demos/08-cl-aws2/demo-2-1-pipeline-s3-dynamodb.md](#)

Un **aws s3 cp**... et 10 produits apparaissent dans l'API, avec un mail de rapport

Ce que la démo assemble

```
aws s3 cp inventaire.csv s3://$PREFIX-.../entrants/
|
v
[ bucket S3 ] -----+
                        v
                        [ Lambda $PREFIX-ingestion ]
                        | valide chaque ligne
                        | batch_writer -> 25 par lot
                        +-----+-----+
                        v               v
[ DynamoDB $PREFIX-stockline ] [ SNS notifications ]
^                               |
|                               v
[ Lambda $PREFIX-produits ]    📧 rapport d'ingestion
^
curl $URL/produits    <- les 10 produits sont là
```



Mini-TP — Pipeline d'ingestion

tp/08-cl-aws2/tp-mini-ingestion.md — 1 h 30, non noté

Vous refaites le pipeline **vous-mêmes**, étape par étape, checkpoints à chaque brique — puis **teardown vérifié**. C'est la répétition générale du TP3.

Récap — les points clés du jour

- **DynamoDB** : items à schéma flexible ; on conçoit la table à **partir des schémas d'accès** ; clé de partition = placement, clé de tri = ordre ; mode à la demande = 0 € à vide.
- **put_item/get_item** par clé, **query** par partition — **scan lit tout et coûte tout** : interdit dans un chemin d'API. Il « marche » en démo : méfiance.
- Pièges boto3 : **Decimal** (pas de float), pagination (**LastEvaluatedKey**).
- **StockLine** : **pk="PRODUIT"/sk=ref** + **pk="MVT#ref"/sk=horodatage** — chaque endpoint servi par get ou query.
- **S3** → **Lambda** : asynchrone, clé encodée URL, permission **avant** notification, préfixes disjoints contre la boucle infinie.
- **SQS** = un travailleur par message, stocke, absorbe (visibility timeout, DLQ, idempotence). **SNS** = tous les abonnés, pousse, ne stocke pas. **Fan-out SNS→SQS** = diffuser puis absorber.



Quiz de fin de journée

10 questions — 15 minutes

Réponses commentées demain matin

Quiz — questions 1 et 2

Question 1 — Votre API doit afficher « les 50 derniers mouvements du produit REF-42 ». Proposez la clé de partition et la clé de tri qui répondent à cette question par une **query**, et écrivez l'appel boto3 correspondant (KeyConditionExpression).

Question 2 — Expliquez pourquoi **scan** avec un **FilterExpression** sur une table de 20 Go coûte le même prix qu'un **scan** sans filtre. À quel moment le filtre s'applique-t-il ?

Quiz — questions 3 et 4

Question 3 — Quelle est la différence entre le mode de capacité **à la demande** et le mode **provisionné** ? Pourquoi avons-nous choisi le premier pour StockLine, et dans quel cas le second devient-il plus intéressant ?

Question 4 — `table.put_item(Item={"pk": "PRODUIT", "sk": "SSD-500", "prix_unitaire": 59.90})` lève `TypeError: Float types are not supported`. Donnez la correction exacte, et expliquez le problème symétrique à la **lecture** des nombres.

Quiz — questions 5 et 6

Question 5 — Deux mouvements de stock sont créés dans la même seconde pour le même produit. Avec `sk = horodatage ISO` seul, que se passe-t-il en base et pourquoi ? Comment notre conception l'évite-t-elle ?

Question 6 — Votre Lambda de miniatures lit les images déposées dans un bucket et écrit les vignettes... dans le même bucket. Le lendemain, la facture Lambda a explosé. Expliquez le mécanisme exact et donnez deux parades.

Quiz — questions 7 et 8

Question 7 — Dans l'événement S3 reçu par une Lambda, le fichier déposé s'appelait `rapport été.csv` mais `event["Records"][0]["s3"]["object"]["key"]` vaut `entrants/rapport+%C3%A9t%C3%A9.csv`, et le `get_object` échoue en `NoSuchKey`. Expliquez et corrigez.

Question 8 — Un consommateur SQS lit un message, le traite en 45 s... et le message est traité une **deuxième fois** par un autre consommateur. Quel réglage est en cause, quelle est sa valeur par défaut, et quelles sont les deux corrections possibles (réglage **et** code) ?

Quiz — questions 9 et 10

Question 9 — Quand un produit passe sous son seuil d'alerte, trois systèmes doivent réagir : l'équipe achats (email), un service de réapprovisionnement (traitement garanti même s'il est en panne 1 h), et un tableau de bord temps réel. Dessinez l'architecture SNS/SQS et justifiez la place de chaque brique.

Question 10 — Dans notre pipeline d'ingestion, le mail SNS n'arrive jamais, alors que les produits apparaissent bien dans DynamoDB et que les logs de la Lambda montrent un `publish` sans erreur. Donnez les deux causes les plus probables, dans l'ordre où vous les vérifieriez.

À demain

Jour 3 — Orchestration et événements

Votre pipeline fonctionne — mais qui **enchaîne** valider → ingérer → notifier → archiver quand ça se complique ? Qui **réessaie** proprement ? Qui remplace le bon vieux cron ?

Demain : **Step Functions** (des workflows dessinés, avec Retry intégré), **EventBridge** (le bus d'événements qui met cron à la retraite), le **tableau de décision honnête** serverless vs EC2/conteneurs — et le schéma complet de la StockLine serverless que vous construirez au TP3. 📖